



Институт экономики и управления
Кафедра статистики и кибернетики

Направление: 09.03.02 Информационные системы и технологии
Направленность:
Компьютерные науки и интеллектуальный анализ данных

КУРСОВОЙ ПРОЕКТ

на тему: «Проектирование и разработка информационной системы
сопровождения выпускных квалификационных работ».

Выполнил:
Студент 3 курса группы Д-Э-311
Эйзерман А. Н.

Дата регистрации КП
на кафедре: _____

Допущен к защите
Руководитель:

учёная степень, учёное звание, ФИО

Члены комиссии:

учёная степень, учёное звание, ФИО

подпись

учёная степень, учёное звание, ФИО

подпись

учёная степень, учёное звание, ФИО

подпись

Оценка _____

Дата защиты _____

Москва 2026

Содержание

Введение	3
Глава 1. Анализ предметной области	5
1.1. Описание процесса сопровождения ВКР	5
1.2. Обзор аналогов и объяснение актуальности работы	7
1.3. Требования к системе	9
1.4. Выбор технологического стека	11
Глава 2. Проектирование системы	13
2.1. Архитектура и паттерн MVC	13
2.2. Проектирование базы данных	14
2.3. Ролевая модель пользователей	16
Глава 3. Разработка системы	19
3.1. Структура Laravel-приложения	19
3.2. Аутентификация и разграничение прав	20
3.3. Реализация основных модулей	21
3.4. Интерфейс	23
3.5. Контейнеризация	25
Заключение	27
Библиографический список	29

Введение

Выпускная квалификационная работа является важнейшим этапом подготовки студента в системе высшего образования. На данной стадии учебного процесса проверяются не только предметные знания, но и умение самостоятельно организовывать работу, соблюдать сроки, взаимодействовать с научным руководителем и оформлять результаты в соответствии с установленными требованиями. В реальной практике сопровождение ВКР включает множество операций, которые должны выполняться последовательно и с фиксацией их результатов. К таким операциям относятся выбор темы, её согласование, закрепление за студентом, контроль стадий выполнения, загрузка итогового документа, передача работы на рассмотрение, выставление оценки и архивирование итогов.

При отсутствии специализированного программного инструмента значительная часть этих действий выполняется вручную. Это приводит к дублированию информации, увеличивает вероятность ошибок в данных и затрудняет контроль текущего состояния работ. Особенно заметными становятся проблемы, связанные с согласованием тем, управлением ролями участников процесса и отслеживанием документов, размещаемых студентами. В подобных условиях возникает потребность в единой информационной системе, которая централизует данные и обеспечивает доступ к ним в зависимости от роли пользователя.

Целью данной курсовой работы является разработка веб-портала для сопровождения выпускных квалификационных работ, который позволяет объединить в одном интерфейсе управление учебными группами, темами ВКР, назначениями, статусами работ и правами пользователей. В качестве технологической основы выбран фреймворк Laravel, так как он предоставляет зрелую архитектуру, встроенные

механизмы маршрутизации, валидации, аутентификации, работы с файлами и реализации модели MVC.

Разработанная система ориентирована на участников учебного процесса с различными функциями. Студент получает доступ к выбору темы, личному кабинету ВКР и документам. Научный руководитель может формировать и предлагать темы, контролировать работы студентов своей группы и просматривать перечень активных назначений. Рецензент и член комиссии получают доступ к тем работам, которые уже находятся на стадии проверки или допускаются к защите. Администратор обладает расширенными полномочиями по управлению пользователями, группами и правами доступа.

Во многих образовательных организациях процессы сопровождения ВКР до сих пор выполняются с использованием нескольких разрозненных инструментов или частично вручную. Это усложняет контроль выполнения работ, приводит к дублированию данных и затрудняет взаимодействие между участниками процесса. Портал, разработанный в рамках данной работы, решает прикладную задачу учёта и контроля, а также формирует основу для дальнейшего расширения функциональности, включая уведомления, отчётность и дополнительные механизмы аналитики.

Глава 1. Анализ предметной области

1.1. Описание процесса сопровождения ВКР

Процесс сопровождения выпускной квалификационной работы можно рассматривать как последовательность управляемых состояний, в рамках которых участвуют несколько категорий пользователей. В центре процесса находится студент, для которого тема работы должна быть либо выбрана из существующего каталога, либо предложена руководителем, либо сформулирована самостоятельно и вынесена на согласование. Вокруг этого сценария строится работа руководителя, кафедры, рецензента и членов комиссии.

На первом этапе система должна обеспечить фиксацию учебной группы. Группа связывает поток студентов с руководителем, годом набора, специальностью и курсом обучения. Руководитель должен видеть только тех студентов и темы, которые относятся к закреплённой за ним группе, а студент должен работать в пределах своей учебной принадлежности.

Следующий этап связан с темами ВКР. В системе тема является самостоятельной сущностью, обладающей автором, типом происхождения, признаком согласования и возможностью резервирования за конкретным студентом. Если тему предлагает студент, она требует согласования и может быть рассмотрена как авторская инициатива. Если тему вносит руководитель, она может быть заранее закреплена за выбранным студентом или оставаться свободной до момента выбора. Если тема формируется администрацией, она попадает в общий каталог и может быть использована несколькими участниками процесса по правилам предметной области.

После утверждения темы создаётся запись о ВКР. Эта запись отражает фактическое состояние работы и хранит историю переходов между

этапами. В карточке работы фиксируются студент, руководитель, учебная группа, тема, способ назначения, даты назначения и ответа, дата начала выполнения, дата сдачи, дата завершения и итоговая оценка. Таким образом, каждая ВКР в системе рассматривается не просто как текстовый документ, а как процесс с жизненным циклом, который поддается учёту и аналитике.

Логика сопровождения ВКР в разработанной системе охватывает несколько типовых сценариев работы. Студент может самостоятельно выбрать тему из согласованного каталога и закрепить её за собой либо получить предложение от руководителя и принять или отклонить его. Для студентов, не определившихся с темой в установленный срок, предусмотрен механизм автоматического распределения. В процессе выполнения работы студент загружает документы, а руководитель, рецензент и комиссия последовательно изменяют статус ВКР в соответствии с этапом проверки. Завершается процесс выставлением итоговой оценки и переводом работы в архивное состояние.

При проектировании процесса важно учитывать не только последовательность действий, но и ограничения. Например, студент не может иметь одновременно несколько активных работ, а тема не должна быть закреплена за двумя разными студентами в одном и том же активном состоянии. Аналогично, предложения руководителя должны обрабатываться в порядке приоритета, чтобы исключить ситуацию, при которой студент получил несколько активных предложений и не может определить, какое из них принять.

С точки зрения управления информацией предметная область характеризуется большим количеством взаимосвязанных сущностей. Каждая операция затрагивает сразу несколько таблиц базы данных и должна выполняться согласованно. Именно поэтому для проекта важны транзакции, правила целостности и чёткое разграничение прав. Без этого система быстро потеряет достоверность, а её данные станут противоречивыми.

Дополнительное значение имеет работа с документами. В процессе выполнения ВКР студент регулярно передаёт промежуточные и итоговые версии файла. Портал должен обеспечивать безопасное хранение

документа, его загрузку в разрешённых форматах и контроль доступа к нему. При этом файл не должен быть доступен посторонним пользователям, а его удаление и замена должны происходить только в допустимой стадии выполнения работы.

Таким образом, предметная область характеризуется сложной логикой согласований, необходимостью учёта ролей и высокой чувствительностью к ошибкам в данных. Это делает разработку специализированной информационной системы оправданной и практически значимой.

1.2. Обзор аналогов и объяснение актуальности работы

На рынке и в образовательной среде существует несколько типов программных решений, которые частично решают задачи сопровождения ВКР. К ним относятся университетские личные кабинеты студентов, универсальные системы электронного документооборота, LMS-платформы, корпоративные порталы и отдельные таблицы, создаваемые кафедрами для внутреннего учёта. Каждое из этих решений покрывает только часть требований, но редко обеспечивает полное соответствие специфике процесса ВКР.

Универсальные учебные платформы обычно хорошо справляются с размещением материалов, тестированием и коммуникацией между преподавателем и студентом. Однако они не всегда ориентированы на жизненный цикл ВКР как отдельного объекта. В них может отсутствовать удобная модель согласования тем, резервирования темы за студентом, фиксации статусов работы или распределения студентов по группам с учётом сроков выбора темы. Если эти механизмы и присутствуют, то нередко реализованы в виде внешних надстроек, что снижает удобство использования.

Системы электронного документооборота, наоборот, обладают развитой инфраструктурой для хранения файлов и утверждения документов, но не учитывают образовательную специфику. В них сложно реализовать понятный для пользователя сценарий работы с темами, учебными группами и ролями участников процесса. Кроме того, подобные решения зачастую избыточны по функциональности и требуют настрой-

ки, которая не оправдана для задачи сопровождения ВКР в рамках одного учебного подразделения.

Отдельные формы, таблицы и локальные реестры, используемые кафедрами, демонстрируют обратную проблему. Они просты в создании, но не обеспечивают достаточного уровня контроля и не позволяют оперативно получать сводную информацию. При увеличении количества студентов и работ такие инструменты становятся неудобными, поскольку требуют ручной проверки совпадений, отслеживания сроков и сверки статусов.

Разработка специализированного портала обусловлена особенностями процесса сопровождения ВКР. В отличие от универсальных систем документооборота, разрабатываемое решение ориентировано на образовательную деятельность и учитывает роли всех участников процесса. Портал обеспечивает хранение истории назначений и изменений статусов, позволяет контролировать ход выполнения работ и предоставляет каждому пользователю только необходимый набор функций. Дополнительным преимуществом является возможность дальнейшего развития системы без существенной переработки существующей архитектуры.

Разработанный портал может использоваться как основа информационной системы кафедры или факультета для сопровождения выпускных квалификационных работ. Он пригоден для демонстрации принципов управления сущностями предметной области, организации прав доступа, работы с файлами и контейнеризации. Кроме того, проект можно рассматривать как базу для последующей интеграции с уведомлениями, календарным планированием, цифровым архивом и аналитическими отчётами.

Наличие аналогов не снижает актуальность собственной разработки. Напротив, сравнение с существующими решениями показывает, что специализированный портал способен обеспечить более точное соответствие реальным процессам сопровождения ВКР.

1.3. Требования к системе

Требования к системе формируются на основе предметной области и набора сценариев, которые должны быть поддержаны программным продуктом. В рассматриваемом проекте требования можно разделить на функциональные и нефункциональные. Такое деление позволяет чётко отделить содержательные возможности системы от технических характеристик её работы.

К функциональным требованиям относятся следующие возможности:

- регистрация, вход в систему и выход из неё;
- восстановление пароля и подтверждение адреса электронной почты;
- поддержка нескольких ролей пользователей и одновременного совмещения ролей;
- создание и редактирование учебных групп;
- назначение руководителя учебной группы;
- создание, редактирование, согласование и просмотр тем ВКР;
- предложение темы студенту руководителем;
- самостоятельный выбор темы студентом из каталога;
- резервирование темы за конкретным студентом;
- создание и ведение записи о ВКР;
- обновление стадий работы и хранение временных отметок;
- загрузка, скачивание и удаление файла ВКР;
- подача, обработка и отклонение заявок на вступление в учебную группу;

- просмотр сводной информации на панели пользователя в зависимости от роли;
- просмотр списка активных работ и канбан-доски статусов.

Нефункциональные требования относятся к качеству и способу реализации системы:

- работа через веб-браузер без установки отдельного клиентского приложения;
- поддержка адаптивной вёрстки для экранов различной ширины;
- надёжное хранение и обработка данных в реляционной базе [2];
- соблюдение правил целостности и ограничений на уровне БД и приложения;
- обеспечение защищённого доступа к данным с учётом ролей пользователей;
- возможность запуска в контейнерной среде;
- поддержка русскоязычного интерфейса и формулировок для образовательного процесса;
- сохранение истории изменений и временных данных.

Для полноценного функционирования системы также важны ограничения предметной области. Например, активная работа у студента должна быть только одна, а тема не может использоваться одновременно несколькими активными назначениями. Если студент имеет незавершённую работу, то новые предложения должны отклоняться либо блокироваться правилами системы. Если тема не согласована, её нельзя использовать в процессе назначения. Если студент уже состоит в группе, повторное вступление должно быть запрещено.

С точки зрения качества интерфейса важно, чтобы система была понятна пользователю без длительного обучения. Это означает, что основные действия должны быть доступны с минимального числа переходов, а статусы должны отображаться наглядно. Для преподавателя важна возможность быстро увидеть список студентов и работ. Для студента важна возможность отследить тему, предложение от руководителя и состояние своего файла. Для администратора важен сводный контроль по группам, темам и пользователям.

Таким образом, требования к системе определяют не только набор экранов и кнопок, но и принципы обработки данных, ограничения целостности и правила доступа. Именно эти требования формируют основу для выбора архитектуры и дальнейшего проектирования.

1.4. Выбор технологического стека

Выбор технологического стека в подобных проектах должен учитывать не только личные предпочтения разработчика, но и особенности задачи, долговременную поддерживаемость и доступность инструментов. Для данного портала был выбран стек, который обеспечивает стабильность, удобство разработки и прозрачность архитектуры.

Основой серверной части является PHP 8.4 и фреймворк Laravel 11. Laravel подходит для образовательных и прикладных систем благодаря структурированному подходу к построению приложения [6]. Он предоставляет маршрутизацию, middleware, системы запросов и ответов, ORM Eloquent, миграции, политики доступа, очереди, файловое хранилище и встроенную поддержку аутентификации. Это позволяет сократить количество вспомогательного кода и сосредоточиться на предметной логике.

Серверная часть Laravel в данном проекте выполняет сразу несколько задач. Она обрабатывает пользовательские запросы, формирует ответы для страниц интерфейса, проверяет права доступа, сохраняет данные в базе и координирует действия, требующие транзакций. Такой подход соответствует классической MVC-модели, в которой контроллер не должен содержать чрезмерно много бизнес-логики, а модель отвеча-

ет за связь с данными и часть правил предметной области.

В качестве системы управления базой данных выбрана MySQL 8.0 [9]. Выбор обусловлен тем, что проект опирается на реляционные связи между сущностями и использует внешние ключи, уникальные ограничения, каскадные операции и индексы [2]. MySQL хорошо подходит для учебного и прикладного проекта, а также легко разворачивается в контейнере Docker. Отдельно важно то, что в проекте используется кодировка utf8mb4, обеспечивающая корректное хранение кириллицы и других Unicode-символов.

Для представления данных пользователю используются Blade-шаблоны. Они позволяют строить серверно-генерируемые страницы и при этом сохранять читаемую структуру интерфейса. Дополнительная стилизация выполнена на Tailwind CSS 4 [12], что упрощает работу с компонентами, карточками и адаптивной сеткой. Vite используется как современный инструмент сборки, обеспечивающий быстрое подключение стилей и JavaScript-модулей [14].

Для интерактивных сценариев применён Alpine.js - минималистичный JavaScript-фреймворк [3][13]. Это особенно оправдано там, где требуется ограниченная логика на клиентской стороне, например перетаскивание карточек на канбан-доске. Использование минимального количества клиентского кода делает систему проще в сопровождении и снижает объём потенциальных ошибок.

Для развёртывания выбран Docker Compose [11]. Данный инструмент позволяет воспроизвести окружение на любом рабочем месте и отделить приложение от системных зависимостей хостовой машины. В состав контейнерной конфигурации входят PHP-FPM, Nginx и MySQL. Такой набор соответствует общепринятому подходу к разворачиванию веб-приложений на базе PHP и делает конфигурацию понятной и повторяемой.

Следовательно, выбранный стек является сбалансированным. Он предоставляет необходимые средства для построения предметно-ориентированного веб-портала и при этом не усложняет проект лишними технологиями, которые не требуются для решаемой задачи.

Глава 2. Проектирование системы

2.1. Архитектура и паттерн MVC

Архитектура проекта построена по принципу разделения ответственности между несколькими слоями [4]. Главным образующим паттерном является MVC, в котором модель отвечает за данные и их связи, представление за отображение, а контроллер за обработку запросов и координацию действий [5]. Для Laravel данная модель особенно естественна, поскольку фреймворк предоставляет инструменты для каждого из указанных слоёв.

Слой моделей в проекте включает сущности User, StudyGroup, Topic, Thesis и StudyGroupJoinRequest. Каждая модель содержит связи Eloquent, вычисляемые свойства и, в ряде случаев, вспомогательные методы для определения состояния объекта. Например, модель Thesis содержит проверку активности, определение завершения и метод завершения работы. Модель User хранит информацию о правах, отображаемом имени, связанных работах и заявках.

Контроллеры сгруппированы по функциональному признаку. В проекте есть контроллеры аутентификации, управления профилем, работы с темами, работами, группами, заявками и административными разделами. Такое разбиение позволяет не смешивать разные области ответственности в одном классе и упрощает сопровождение кода. Например, действия с темами находятся в TopicController, а обновление статуса работы и загрузка документов вынесены в ThesisController.

Помимо базовой схемы MVC в проекте используются дополнительные архитектурные механизмы. Для контроля доступа к темам и ВКР применяются Policies, а для определения глобальных ролей пользователей используются Gates. Сложные сценарии обработки данных вынесены в сервисные классы, что позволяет сохранять контроллеры компакт-

ными. Для представления фиксированных наборов значений используются перечисления Enums, а повторяющиеся элементы интерфейса реализованы с помощью Blade-компонентов.

Такой подход приближается к дисциплинированной многослойной архитектуре. Контроллеры не хранят всю бизнес-логику, а модели не превращаются в пассивные контейнеры данных. В результате код проще читать, тестировать и дорабатывать.

Если рассматривать проект в терминах слоёв, то можно выделить следующие уровни взаимодействия. Сначала HTTP-запрос приходит в маршрут, затем попадает в контроллер, после чего при необходимости обращается к политике, сервису и модели. Модель, в свою очередь, работает с базой данных через Eloquent. Ответ формируется в шаблоне Blade и возвращается пользователю. Такая цепочка хорошо соответствует веб-приложению с ролевым интерфейсом.

Особую важность для архитектуры имеет соблюдение границы между представлением и данными. В шаблонах отображаются уже подготовленные значения, а не реализуется сложная логика вычислений. Это снижает вероятность ошибок и делает код интерфейса более понятным. При этом интерактивные элементы, такие как канбан-доска, не нарушают общую архитектуру, так как они работают только с локальным состоянием DOM и отправляют на сервер минимально необходимый запрос.

В итоге использование MVC в данном проекте нельзя считать формальным. Оно действительно определяет структуру приложения и позволяет распределить функции между слоями таким образом, чтобы каждый компонент выполнял свою задачу.

2.2. Проектирование базы данных

База данных является основой всей системы, поскольку именно она хранит сведения о пользователях, группах, темах, работах и заявках. Структура БД строилась исходя из принципов нормализации и целостности данных [2]. Это позволило минимизировать дублирование информации и сохранить логическую непротиворечивость.

Главная таблица `users` содержит учётные записи пользователей, включая фамилию, имя, отчество, адрес электронной почты, пароль, признаки подтверждения и поле прав `permissions`. Дополнительно к ней привязана ссылка на учебную группу. Такое решение позволяет хранить пользователя как универсальную сущность, которая может исполнять несколько ролей одновременно.

Таблица `study_groups` описывает учебные группы. В ней хранятся название группы, курс, код и наименование специальности, руководитель группы, год набора и крайний срок выбора темы. Сущность группы связывает студентов с организационной структурой процесса ВКР. Наличие даты дедлайна позволяет автоматически ограничивать сценарии назначения тем и запускать случайное распределение после истечения срока.

Таблица `topics` отвечает за темы выпускных работ. В ней фиксируются название, описание, автор, тип темы, резервирование за студентом и признаки согласования. У темы может быть автор в лице студента или преподавателя, что поддерживает два различных сценария формирования каталога. Поля согласования позволяют отделить созданную тему от той, которая уже готова к использованию.

Таблица `theses` является центральной с точки зрения сопровождения работы. Именно в ней фиксируются все события жизненного цикла ВКР. Сюда входят студент, руководитель, группа, тема, способ назначения, статус согласования, статусы выполнения и временные отметки. Дополнительно хранится путь к загруженному файлу и его оригинальное имя, что позволяет работать с документом как с материальным объектом процесса.

Таблица `study_group_join_requests` предназначена для фиксации заявок на вступление студента в группу. Её наличие важно, поскольку в образовательном процессе часто требуется отдельная проверка и подтверждение принадлежности студента к группе. Заявка хранит статус, обработавшего пользователя и момент обработки. Это обеспечивает трассируемость управленческого действия.

С точки зрения структуры данных пользователь может принадлежать одной учебной группе. Каждая группа связана с руководителем,

который также представлен сущностью пользователя, и содержит множество студентов и связанных с ними записей о ВКР. Темы могут участвовать в истории назначений, однако активным в каждый момент времени остаётся только одно назначение. Заявка на вступление используется для связи конкретного студента с выбранной учебной группой и фиксации процесса зачисления.

Для поддержания производительности в таблице `theses` добавлены индексы по сочетаниям `student_id` и `done_at`, `study_group_id` и `assignment_status`, а также `supervisor_id` и `assignment_status`. Такие индексы ускоряют выборку активных и связанных работ, что особенно важно для панели руководителя и общего списка ВКР.

Дополнительную роль в схеме играют ограничения внешних ключей. Некоторые связи настроены с `restrictOnDelete`, чтобы исключить случайное удаление сущностей, на которые ссылаются активные записи. В других случаях используется `nullOnDelete`, если необходимо сохранить историю, но убрать ссылку на удалённую сущность. Это соответствует логике образовательного учёта, где исторические данные часто должны оставаться доступными.

Проектирование базы данных в данном случае можно считать достаточно строгим, поскольку схема не только описывает структуру хранения, но и участвует в реализации бизнес-правил. За счёт этого уменьшается вероятность нарушения целостности, даже если какая-либо часть логики на уровне приложения будет изменена в будущем.

2.3. Ролевая модель пользователей

Ролевая модель является одним из ключевых элементов системы, поскольку именно она определяет доступность операций и видимость разделов интерфейса. В проекте роли реализованы не как одно из взаимоисключающих значений, а как набор битовых флагов. Это означает, что один пользователь может сочетать несколько ролей одновременно. Такой подход особенно полезен в образовательной среде, где преподаватель может одновременно быть руководителем, рецензентом или членом комиссии.

В модели User определены следующие флаги:

- PERM_STUDENT - для студента;
- PERM_SUPERVISOR - для преподавателя-руководителя;
- PERM_REVIEWER - для рецензента;
- PERM_COMMISSION - для члена комиссии;
- PERM_ADMIN - для администратора.

На уровне модели предусмотрены методы проверки и изменения прав. Это позволяет не дублировать побитовые операции по всему коду, а использовать понятные высокоуровневые вызовы. Например, можно проверить, является ли пользователь администратором, или добавить ему новую роль без изменения логики всех контроллеров.

Гибкость ролей важна и с точки зрения пользовательских сценариев. Студент видит только те разделы, которые ему действительно нужны. Руководитель получает доступ к темам, группам и активным работам своих студентов. Комиссия и рецензент видят только релевантные записи в нужных статусах. Администратор может управлять всеми сущностями и, при необходимости, корректировать права пользователей.

Разграничение доступа построено на нескольких уровнях. Глобальные проверки через Gate быстро определяют принадлежность к ключевой роли. Политики TopicPolicy и ThesisPolicy проверяют права на конкретный объект. Сервис StudyGroupMembershipService дополнительно убеждается, что студент может быть добавлен только в допустимую группу, а руководитель управляет только своей областью ответственности.

Такое сочетание ролей и политик формирует устойчивую систему доступа. Пользователь не только не может выполнить запрещённое действие, но и не видит лишние операции в интерфейсе. Это снижает риск ошибок и делает приложение удобным для повседневной работы.

Особого внимания заслуживает сценарий первого администратора. В системе предусмотрена возможность автоматического создания

или обновления административной учётной записи через специальную консольную команду. Кроме того, при первоначальной регистрации и наличии токена регистрации новый пользователь может получить роль администратора. Это полезно для разворачивания системы в учебной или демонстрационной среде.

В итоге ролевая модель в проекте выполняет не вспомогательную, а структурообразующую функцию. Она связывает бизнес-логику, интерфейс и уровень доступа к данным в единую систему правил.

Глава 3. Разработка системы

3.1. Структура Laravel-приложения

Структура Laravel-приложения в проекте соответствует стандартному подходу фреймворка и одновременно отражает предметную логику. В каталоге `app` находятся модели, контроллеры, политики, сервисы, перечисления и консольные команды. В `routes` описаны маршруты веб-интерфейса и аутентификации. В `resources/views` расположены шаблоны экранов, а в `resources/js` и `resources/css` реализована клиентская интерактивность и визуальное оформление.

Каталог `app/Models` содержит основные сущности предметной области. Каждая модель не только описывает таблицу базы данных, но и предоставляет связи с другими сущностями, вычисляемые свойства и методы, необходимые для проверки статуса объекта. Например, пользователь знает о своих темах и работах, а тема знает о связанных назначениях и авторе. Это удобно, поскольку большая часть логики может быть сформулирована через Eloquent-связи.

Контроллеры сгруппированы по функциональному признаку. В проекте есть контроллеры аутентификации, управления профилем, работы с темами, работами, группами, заявками и административными разделами. Такое разбиение позволяет быстро находить нужную точку входа и не смешивать действия, которые относятся к разным ролям и разным страницам.

Маршруты также организованы по логике использования. Пользовательские действия доступны только после аутентификации, а административные страницы дополнительно защищены проверкой `can:admin`. Публичным остаётся только минимальный набор страниц, связанный с входом и регистрацией. Подобный подход соответствует принципу минимизации открытой поверхности приложения.

Важную часть структуры составляют перечисления статусов и типов. Они находятся в каталоге `app/Enums` и используются для темы, ВКР, значения и заявок. Применение перечислений повышает читаемость кода и устраняет множество ошибок, связанных с использованием строковых или числовых литералов без централизованного описания.

Слой представления построен на Blade-компонентах. В проекте используются общие компоненты для кнопок, полей, бейджей, модальных окон, навигации и отдельных форм. Это позволяет повторно использовать одинаковые элементы и поддерживать единый визуальный язык на всех страницах.

Можно отметить, что структура проекта ориентирована не только на соответствие шаблону `Laravel`, но и на удобство сопровождения [1]. Это важно для учебной разработки, поскольку такой код проще анализировать, демонстрировать и дорабатывать в рамках следующих итераций.

3.2. Аутентификация и разграничение прав

Система аутентификации построена на механизмах, предоставляемых `Laravel` [8]. Пользователь может зарегистрироваться, войти в систему, восстановить пароль, подтвердить электронную почту и завершить сеанс работы. Этот набор функций покрывает базовые потребности веб-приложения и избавляет от необходимости писать уязвимую самодельную реализацию [7].

Регистрация реализована так, чтобы учитывать не только обычного пользователя, но и сценарий создания первого администратора. Если в конфигурации задан регистрационный токен и в системе ещё нет администратора, форма регистрации может предоставить повышенные права новому пользователю. Такое решение удобно для первичного запуска проекта, поскольку позволяет не выполнять отдельное ручное назначение роли через базу данных.

В проекте реализовано несколько уровней разграничения доступа. На верхнем уровне используются Gate, которые определяют принадлежность к администраторам, руководителям, комиссии и рецензентам. На уровне конкретных объектов используются политики. Напри-

мер, тема может редактироваться только её автором или администратором, и только если она ещё не занята активной работой. ВКР можно просматривать более широкому кругу пользователей, но изменять её статус способны только участники, обладающие соответствующим правом.

Отдельно следует отметить сервисный уровень. Класс `StudyGroupMembershipService` объединяет операции добавления студента в группу, обработки заявок, отклонения заявок и удаления участника. Этот сервис сам проверяет, является ли пользователь студентом, не состоит ли он уже в другой группе и может ли текущий исполнитель управлять данной группой. Подобная централизация правовых проверок уменьшает риск рассинхронизации логики.

В модели доступа важную роль играет связь между ролями и состоянием объекта. Пользователь не только должен обладать формальным правом, но и должен находиться в правильном контексте. Например, руководитель может управлять только теми группами, за которые он отвечает, а комиссия видит только те работы, которые уже достигли соответствующих стадий. Это показывает, что авторизация в проекте строится не только по принципу «кто вы», но и по принципу «какой объект вы пытаетесь изменить».

Правильность разграничения прав поддерживается и на уровне пользовательского интерфейса. В навигации и на страницах отображаются только доступные действия. Это снижает количество ошибок, повышает удобство работы и делает поведение системы предсказуемым. При этом скрытие кнопки не заменяет серверную проверку, а лишь дополняет её.

Таким образом, аутентификация и авторизация в данном проекте представляют собой комплексную подсистему, включающую конфигурацию, маршруты, политики, сервисы и интерфейсные ограничения.

3.3. Реализация основных модулей

Функциональная часть проекта разделена на несколько крупных модулей, каждый из которых обслуживает собственный набор сценариев.

Такое деление помогает понять систему как совокупность взаимосвязанных подсистем, а не как набор разрозненных страниц.

Модуль учебных групп реализует создание, редактирование и просмотр групп. Администратор может заводить новые группы, назначать руководителя, указывать специальность и дедлайн выбора темы. Руководитель группы может просматривать состав студентов, заявки на вступление и активные ВКР. Внутри этого же сценария предусмотрено случайное распределение тем после истечения дедлайна, если студенты не сделали выбор самостоятельно.

Модуль тем ВКР отвечает за весь жизненный цикл темы. Пользователь может создать тему, просмотреть её карточку, изменить описание и инициировать согласование. Студент имеет право предложить собственную тему, а руководитель может предложить тему конкретному студенту. После согласования тема попадает в общий список доступных тем, откуда студент может её закрепить. Также предусмотрено резервирование темы, если руководитель заранее связывает её с конкретным студентом.

Модуль работ является центральным для учёта выполнения ВКР. Он хранит карточку работы, отображает её статус, позволяет загрузить документ и фиксирует важные временные отметки. Внутри модуля реализованы сценарии принятия и отклонения предложения руководителя, обновления стадии работы и перевода записи в завершённое состояние. При завершении работы дополнительно сохраняется итоговая оценка.

Модуль заявок на вступление в группу позволяет студенту отправить запрос на закрепление за учебной группой. После этого руководитель или администратор может обработать заявку, одобрить её либо отклонить. Такой механизм полезен, когда студент переводится, только что зарегистрировался или ещё не закреплён за группой, но должен быть включён в учебный поток.

Административный модуль обеспечивает управление пользователями и группами. В нём можно изменить данные пользователя, назначить роли, прикрепить или открепить от группы, а также просматривать подробную карточку участника процесса. Это особенно важно для ситуаций, когда права и принадлежность к группе изменяются в тече-

ние семестра.

Каждый модуль построен так, чтобы выполнять понятную предметную задачу и при этом не требовать излишнего ручного вмешательства. Например, при принятии предложения темы система автоматически снимает резервирование с темы, обновляет статус назначения и переводит работу в активное состояние. При случайном распределении заранее закрываются активные предложения, чтобы не возникали дублирующие записи. При обработке заявки в группу автоматически обновляются сопутствующие связи.

Отдельное внимание заслуживает работа с временными отметками. В модели Thesis хранится не только текущий статус, но и последовательность важных дат. Это позволяет не просто видеть конечное состояние работы, а восстанавливать хронологию процесса. Для учебной системы это особенно ценно, поскольку позволяет анализировать дисциплину выполнения и скорость прохождения этапов.

Таким образом, основные модули реализуют полный цикл сопровождения ВКР, начиная с темы и группы и заканчивая итогом защиты и архивированием работы.

3.4. Интерфейс

Интерфейс проекта построен на Blade-шаблонах и оформлен средствами Tailwind CSS. Основной задачей при разработке интерфейса было не создать декоративную оболочку, а обеспечить понятную и функциональную среду для студентов, преподавателей и администраторов. По этой причине структура страниц строится вокруг карточек, таблиц, бейджей статуса и чёткой навигации.

Главная навигация отображает доступные разделы в зависимости от роли пользователя. Студент видит панель, темы, личную работу и профиль. Руководитель дополнительно получает доступ к списку работ и к своей группе. Администратор видит разделы управления группами и пользователями. Такая логика делает интерфейс компактным и снижает визуальную перегрузку.

Панель студента ориентирована на повседневные действия. На ней

отображается текущая работа, доступные темы, предложения руководителя, собственные темы и информация о группе. Пользователь может сразу увидеть, есть ли у него активная ВКР, поступили ли новые предложения и какие темы находятся в обработке. Это сокращает число переходов и делает личный кабинет действительно рабочим инструментом.

Панель преподавателя построена вокруг контроля над группами и работами. На ней представлены закреплённые группы, ожидающие ответа предложения, активные работы и список собственных тем. Для руководителя это позволяет быстро оценить состояние группы, понять, где есть незавершённые предложения, и перейти к нужной работе без поиска по множеству страниц.

Интерфейс комиссии и рецензентов предельно сфокусирован на просмотре работ, находящихся в нужных статусах. Это позволяет не перегружать пользователей лишней информацией. В админ-панели, напротив, собрана сводная информация о количестве активных работ, тем и групп, а также последние созданные сущности. Такой экран полезен для общего контроля и административного мониторинга.

Страницы тем и работ реализованы как подробные карточки. В них отображаются автор, согласующий, резервирование, история назначений, временная линия и статус. На странице ВКР предусмотрен блок загрузки документа и блок изменения статуса, если текущий пользователь обладает соответствующими правами. Это делает карточку сущности не просто просмотром данных, а местом выполнения основных действий.

Особенностью интерфейса является наличие канбан-доски на странице работ. Карточки ВКР располагаются по колонкам статусов и могут перемещаться между ними. При перетаскивании система выполняет запрос к серверу и сохраняет новое состояние. Если операция не удалась, карточка возвращается назад. Такой механизм удобен для быстрого управления статусами и делает экран более наглядным.

Оформление страниц выполнено в спокойной светлой гамме с нейтральными оттенками. Это соответствует образовательному характеру системы, не отвлекает пользователя и позволяет долго работать с интерфейсом без визуального напряжения. Компонентная структура

Blade-шаблонов упрощает повторное использование карточек, форм и кнопок, а также поддерживает единый стиль на всех страницах.

Следовательно, интерфейс проекта можно считать достаточно зрелым с точки зрения пользовательского опыта.

3.5. Контейнеризация

Контейнеризация является важным элементом современного веб-проекта, поскольку позволяет получить воспроизводимое окружение и избежать расхождений между локальной разработкой, тестированием и запуском на сервере. В данном проекте используется Docker Compose [11], который объединяет несколько сервисов в единый сценарий запуска.

В состав контейнерного окружения входят три основных сервиса. Первый сервис `app` содержит PHP-FPM, Composer и Node.js. Второй сервис `nginx` обслуживает HTTP-запросы и передаёт PHP-файлы на обработку в контейнер приложения [10]. Третий сервис `db` предоставляет MySQL 8.0 и хранит данные в именованном томе. Такой набор является минимально достаточным для полноценной работы Laravel-приложения.

Dockerfile устанавливает необходимые системные библиотеки и PHP-расширения, включая модули для работы с базой данных, строками и математическими операциями. Затем в контейнер добавляется Composer, после чего подключаются скрипты и настраивается рабочая директория. Отдельный `entrypoint` выполняет подготовку прав доступа к `storage` и `bootstrap/cache`, после чего запускается PHP-FPM.

Сценарий первоначальной настройки вынесен в отдельный скрипт `setup.sh`. Он устанавливает зависимости PHP и JavaScript, создаёт файл окружения при его отсутствии, генерирует ключ приложения, выполняет миграции, создаёт администратора и собирает фронтенд. Наличие такого сценария упрощает запуск проекта на новом компьютере и делает процесс развёртывания предсказуемым.

Важной деталью контейнерной конфигурации является настройка кодировки MySQL на `utf8mb4`. Это гарантирует корректную обработку

русскоязычных данных, имён пользователей, тем и описаний. Для учебной системы на русском языке это принципиально важное требование, поскольку любая ошибка в кодировке может нарушить корректность отображения и поиска.

Nginx в проекте используется как лёгкий и понятный обратный прокси [10]. Он принимает запросы на 80 порту, обслуживает статические файлы и передаёт PHP-скрипты в приложение. Такая схема является стандартной для Laravel и хорошо подходит для локальной эксплуатации, поскольку не требует сложной инфраструктуры.

Использование контейнеризации позволяет обеспечить одинаковое окружение независимо от компьютера, на котором запускается проект. Благодаря Docker упрощается установка зависимостей и первичная настройка системы, исключаются конфликты между версиями PHP, Node.js и MySQL, а также облегчается перенос приложения между различными средами эксплуатации.

Таким образом, контейнеризация в проекте выполняет прикладную функцию. Она является частью архитектуры развёртывания и делает проект более устойчивым к внешним различиям среды.

Заключение

В рамках курсовой работы был разработан веб-портал сопровождения выпускных квалификационных работ, предназначенный для использования в образовательном процессе. Система объединяет управление учебными группами, темами, назначениями и статусами ВКР, а также поддерживает разграничение прав между студентами, преподавателями, рецензентами, членами комиссии и администраторами.

В процессе выполнения работы была проанализирована предметная область, определены основные сценарии работы с темами и ВКР, сформулированы требования к системе и выбран технологический стек, соответствующий этим требованиям. На стадии проектирования была разработана реляционная схема базы данных, определена ролевая модель пользователей и заложены правила разграничения доступа.

Практическая реализация показала, что Laravel является удобной платформой для построения подобных систем, так как он сочетает архитектурную упорядоченность, готовые механизмы аутентификации и богатые средства работы с данными. Дополнительные элементы, такие как Policies, Services, Enums и Blade-компоненты, позволили сделать код более структурированным и ближе к требованиям реального приложения.

Интерфейс системы был реализован в виде набора адаптивных страниц, ориентированных на разные роли пользователей. Наиболее важные сценарии, включая выбор темы, предложение темы студенту, загрузку документа, изменение статуса работы и обработку заявок на вступление в группу, были сведены к удобным и наглядным действиям. Контейнеризация обеспечила воспроизводимость окружения и упростила развёртывание проекта.

Разработанный портал можно рассматривать как основу для даль-

нейшего развития. Перспективными направлениями расширения являются уведомления о событиях, расширенная аналитика по срокам и статусам, интеграция с электронным архивом документов, а также подключение дополнительных механизмов отчётности для кафедры и деканата. Таким образом, цель курсовой работы достигнута, а созданное программное решение обладает практической ценностью и потенциалом дальнейшего развития.

Список использованных источников

1. Мартин, Р. Чистый код. Создание, анализ и рефакторинг / Р. Мартин. - СПб.: Питер, 2019. - 464 с.
2. Дейт, К.Дж. Введение в системы баз данных / К.Дж. Дейт. - М.: Вильямс, 2006. - 1328 с.
3. Флэнаган, Д. JavaScript: подробное руководство / Д. Флэнаган. - СПб.: БХВ-Петербург, 2012. - 1080 с.
4. Мартин, Р. Чистая архитектура: искусство разработки программного обеспечения / Р. Мартин. - СПб.: Питер, 2018. - 352 с.
5. Фаулер, М. Шаблоны корпоративных приложений / М. Фаулер. - М.: Вильямс, 2012. - 544 с.
6. Отт, Т. Laravel: Up & Running. A Framework for Building Modern PHP Apps / Т. Отт. - Sebastopol: O'Reilly Media, 2023. - 620 с.
7. Скафф, Б. PHP 8. Объекты, шаблоны и методики программирования / Б. Скафф. - М.: Вильямс, 2022. - 736 с.
8. Официальная документация Laravel 11 [Электронный ресурс]. - Режим доступа: <https://laravel.com/docs/11.x> (дата обращения: 01.06.2025).
9. Официальная документация MySQL 8.0 [Электронный ресурс]. - Режим доступа: <https://dev.mysql.com/doc/refman/8.0/en/> (дата обращения: 01.06.2025).
10. Официальная документация Nginx [Электронный ресурс]. - Режим доступа: <https://nginx.org/ru/docs/> (дата обращения: 01.06.2025).

11. Официальная документация Docker Compose [Электронный ресурс].
- Режим доступа: <https://docs.docker.com/compose/> (дата обращения: 01.06.2025).
12. Официальная документация Tailwind CSS 4 [Электронный ресурс].
- Режим доступа: <https://tailwindcss.com/docs> (дата обращения: 01.06.2025).
13. Официальная документация Alpine.js [Электронный ресурс]. - Режим доступа: <https://alpinejs.dev/docs> (дата обращения: 01.06.2025).
14. Официальная документация Vite [Электронный ресурс]. - Режим доступа: <https://vite.dev/guide/> (дата обращения: 01.06.2025).